



**Universidad Tecnológica del Perú**

**“Año de la Esperanza y el Fortalecimiento de la  
Democracia” Facultad de Ingeniería de Sistemas e  
Informática**

**Diseño de Productos y Servicios**

## **Sistema IA para priorización de emergencias**

Bustamante Soria, Fabrizio Alessandro – 100%

Garcia Ortega, Shayuri Kiara- 100%

Mantari Licapa, Walter Isaac – 100%

Ramirez Rivera, Adrian Jose Felix – 100%

Rodriguez Munaylla, Marcela Natalie – 100%

Asistente Virtual: Emergenc- IA – 100%

Guevara Jimenez Jorge Alfredo

**2026 – Lima**

# Caso de estudio

Una central de monitoreo municipal (Comas) requiere una solución tecnológica de gestión de emergencias centralizada que permita:

- Reporte inmediato con captura automática de ubicación GPS vía navegador.
  - Reporte asistido por texto para ciudadanos con discapacidades o situaciones de alto estrés.
  - Clasificación automática del tipo de incidente mediante Inteligencia Artificial (NLP).
  - Despacho y sincronización en tiempo real mediante APIs externas con Serenazgo, PNP y Bomberos.
  - Panel centralizado para el operador con acceso restringido por roles.
- 

## Historia de usuario principal

**Como** ciudadano en situación de riesgo,  
**quiero** disponer de un portal web con un chatbot interactivo y botones de pánico,  
**para** reportar mi emergencia de forma rápida y sin fricciones.

---

## Criterios de aceptación (Given–When–Then)

### Escenario 1: Envío de alerta inmediata mediante Botón de Pánico

**Given** que el ciudadano posee un dispositivo con acceso a internet y se encuentra en una situación de alto estrés.

**When** el ciudadano presiona el botón de pánico central en el portal web.

**Then** el sistema captura instantáneamente su ubicación GPS a través del navegador y envía la alerta priorizada directamente a la central de monitoreo.

---

## Escenario 2: Integración con establecimiento externo

**Given** que un ciudadano con discapacidad auditiva o del habla necesita reportar una emergencia sin usar la voz.

**When** el ciudadano inicia una conversación de texto con el chatbot asistente del sistema.

**Then** el sistema procesa la descripción mediante Inteligencia Artificial, guía al usuario paso a paso para recolectar los datos vitales y clasifica automáticamente el tipo de incidente.

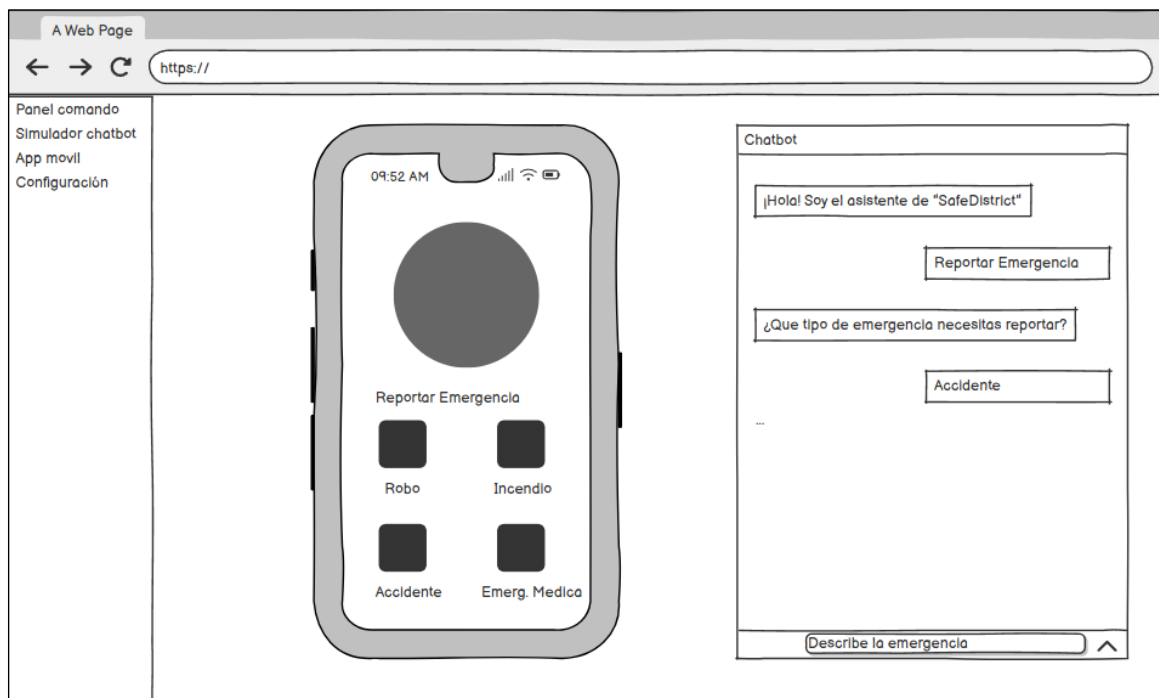
---

## Escenario 3: Apoyo de IA

**Given** que el sistema ha clasificado de forma automática un incidente crítico en la base de datos centralizada.

**When** el operador de la central valida el reporte visualmente en su dashboard.

**Then** el sistema integra y activa los protocolos de interoperabilidad técnica para despachar y sincronizar los recursos en tiempo real con Serenazgo, la PNP o los Bomberos.



# Actividad 1 – Análisis funcional y definición de la prueba de concepto

## Propósito

Comprender la historia de usuario principal y definir los componentes funcionales mínimos de la prueba de concepto.

## Producto esperado

Tabla de análisis funcional.

| Elemento           | Descripción                                                                                          |
|--------------------|------------------------------------------------------------------------------------------------------|
| Actor principal    | Ciudadano en situación de riesgo (Poblador de Comas).                                                |
| Actor secundario   | Operador de la central de emergencias / Serenazgo.                                                   |
| Servicio externo   | API de Geolocalización del Navegador (GPS).                                                          |
| Servicio externo   | APIs de entidades de respuesta (Serenazgo, PNP, Bomberos).                                           |
| Servicio externo   | Núcleo de Inteligencia Artificial (Procesamiento de Lenguaje Natural - NLP).                         |
| Función principal  | Reporte inmediato mediante Botón de Pánico con captura automática de ubicación.                      |
| Función secundaria | Interacción interactiva y triaje automatizado por Chatbot Web.                                       |
| Restricción        | Interfaz multicanal accesible, responsiva y de baja fricción sin descargas obligatorias de software. |

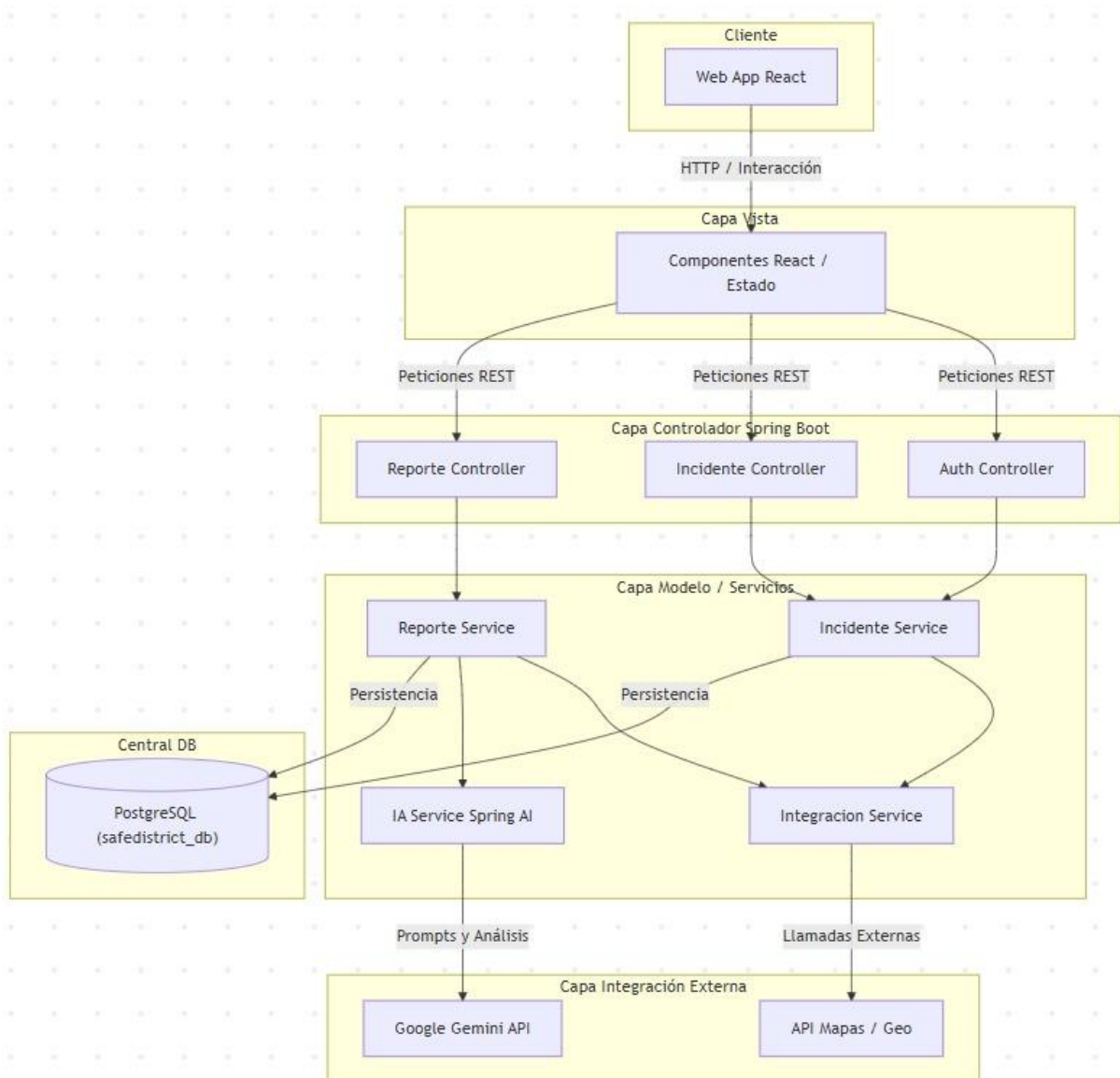
---

# Actividad 2 – Diseño de arquitectura de software en capas

## Propósito

Diseñar la arquitectura base de la solución usando MVC, base de datos e integración con IA y servicios externos.

## Arquitectura propuesta



## Explicación breve de las capas

- **Cliente:** Web App (Vite + React) con el Dashboard Táctico y el chatbot de triaje ciudadano.
  - **Capa Vista:** Componentes visuales de React (IncidentMap) y manejo del estado global.
  - **Capa Controlador:** Endpoints RESTful de Spring Boot que exponen funcionalidades (ReporteController, IncidenteController).
  - **Capa Modelo / Servicios:** Lógica de negocio, gestión de entidades JPA y orquestación con la IA.
  - **Capa Integración Externa:** Llamadas a servicios de terceros, destacando la Google GenAI API (Gemini) y servicios geoespaciales.
  - **Central DB:** Base de datos PostgreSQL para la persistencia
- 

## Flujo de Integración Identificado (con IA)

- **Recepción:** El ciudadano envía un reporte desde la Web App.
  - **Petición:** Pasa al ReporteController y luego al ReporteService.
  - **Análisis (Integración Externa):** El IA Service envía la descripción a la API de Google Gemini para evaluar la urgencia.
  - **Respuesta y Persistencia:** Gemini clasifica la emergencia, y el backend la guarda en PostgreSQL.
  - **Actualización Táctica:** El Dashboard del operador consume estos nuevos datos para renderizarlos en el mapa en tiempo real.
-

# Actividad 3 – Organización del repositorio

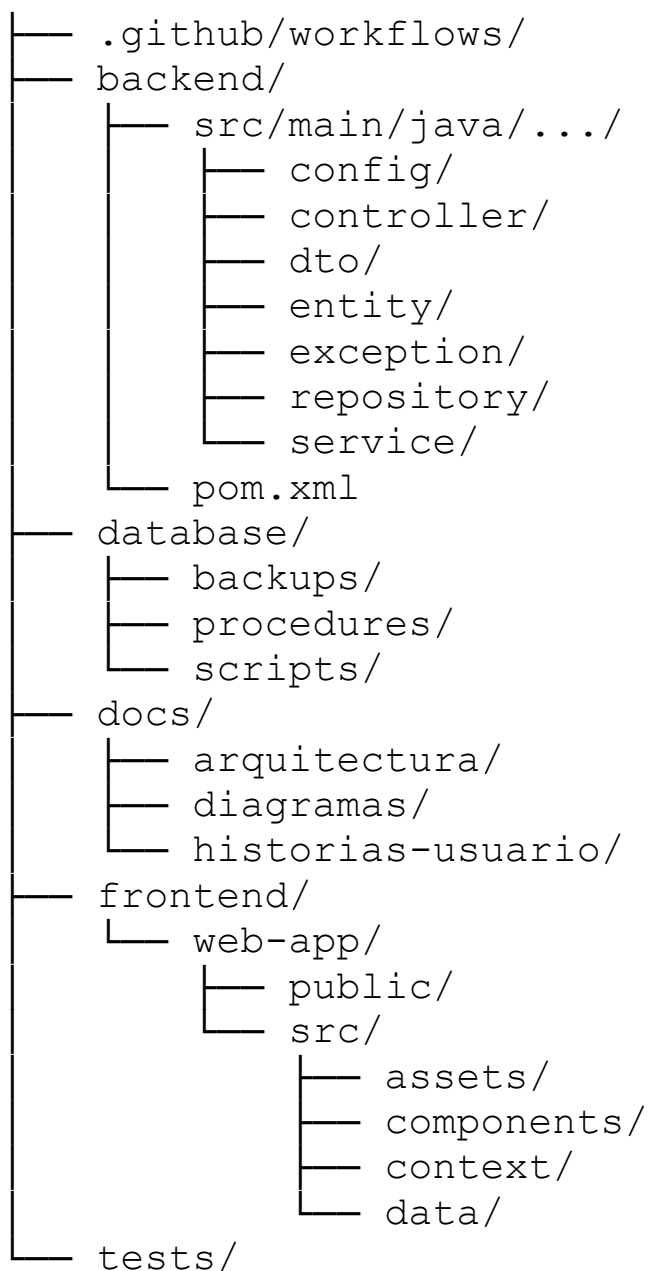
## GitHub y selección tecnológica

### Propósito

Definir la estructura inicial del proyecto y tecnologías alternativas por capa.

---

### Estructura propuesta del repositorio GitHub



# Herramientas alternativas por capa

## Capa Frontend

| Herramienta | Tipo |
|-------------|------|
|-------------|------|

|       |     |
|-------|-----|
| React | Web |
|-------|-----|

---

## Capa Backend

| Herramienta | Tipo |
|-------------|------|
|-------------|------|

|               |      |
|---------------|------|
| Spring Boot 3 | Java |
|---------------|------|

---

## Base de datos

| Herramienta | Tipo |
|-------------|------|
|-------------|------|

|            |            |
|------------|------------|
| PostgreSQL | Relacional |
|------------|------------|

---

## Herramientas de integración y APIs

| Herramienta | Uso |
|-------------|-----|
|-------------|-----|

|         |             |
|---------|-------------|
| Postman | Pruebas API |
|---------|-------------|

|         |               |
|---------|---------------|
| Swagger | Documentación |
|---------|---------------|

---

## Preguntas de reflexión

1. ¿Cómo contribuye MVC a la escalabilidad del sistema?

El patrón MVC separa limpiamente la interfaz de usuario (Vista) de la lógica del negocio (Modelo) y de las rutas de comunicación (Controlador). Esto permite escalar los componentes de forma independiente; por ejemplo, si hay un pico masivo de reportes de emergencia de ciudadanos, se puede aumentar la capacidad de los servidores Backend y de la Base de Datos sin necesidad de modificar o reinstalar la aplicación que el ciudadano visualiza en su navegador.

2. ¿Qué riesgos existen en la integración entre entidades externas?

Los riesgos principales incluyen la latencia o caída del servicio de las APIs externas (por ejemplo, si la API de la PNP no responde a tiempo), problemas de



incompatibilidad de formatos de datos (sistemas heterogéneos) y brechas de seguridad, donde la interconexión podría exponer datos vulnerables de los ciudadanos si los canales de comunicación no están cifrados adecuadamente.

### 3. ¿Qué ventajas aporta la IA en la atención?

Permite automatizar el triaje inicial interactuando con miles de ciudadanos simultáneamente mediante lenguaje natural. La IA es capaz de analizar el texto de forma inmediata, clasificar el nivel de severidad del incidente y categorizarlo automáticamente (ej. derivar directamente a Bomberos si detecta palabras clave de incendios), agilizando drásticamente los tiempos de respuesta humana en la central.

### 4. ¿Cómo se garantiza la interoperabilidad entre sistemas heterogéneos?

Se garantiza mediante la implementación de estándares abiertos de comunicación universal, principalmente a través del diseño de **APIs RESTful** estandarizadas que utilicen formatos de intercambio de datos comunes como **JSON**. Adicionalmente, el uso de herramientas de documentación interoperable como *Swagger* asegura que cualquier entidad externa sepa exactamente cómo enviar y recibir datos del sistema central sin importar qué lenguaje de programación utilicen internamente

---

---

# Actividad Final – Tarea para próximos pasos del proyecto

## Propósito

Extender la prueba de concepto hacia una implementación funcional completa de la historia de usuario principal usando todas las capas de software.

## Indicaciones de desarrollo

Cada equipo deberá desarrollar la historia de usuario en las siguientes capas:

### 1. Frontend

- Formulario web dinámico y widget de chatbot accesible para el ciudadano.
- Botón de pánico táctil de alta visibilidad para reportes expresos.
- Dashboard interactivo para la visualización de alertas prioritarias del operador.
- Interfaz responsiva adaptada a dispositivos móviles y navegadores web.

### 2. Backend

- API REST funcional para la recepción de alertas, coordenadas GPS y mensajes.
- Controladores MVC estructurados para la gestión del flujo del reporte.
- Validaciones de negocio para mitigar el ruido y la saturación del sistema.

### 3. Capa de servicios

- Integración simulada con la API de geolocalización automática por navegador.
- Integración simulada con el núcleo de Inteligencia Artificial (NLP) para el triaje y clasificación del chat.
- Integración con protocolos externos para el despacho unificado a las unidades (Serenazgo, PNP y Bomberos).

### 4. Base de datos

- Modelo entidad-relación para el almacenamiento del historial de incidentes y métricas.
- Scripts SQL para la inicialización y estructuración de tablas centralizadas.

- Procedimientos almacenados para el cálculo automatizado de tiempos de triaje y KPIs operativos.

## **5. Seguridad**

- Login básico para el acceso de los operadores municipales.
- Validación de roles jerárquicos (Operador de central, Jefe de unidad en campo, Administrador).
- Control de acceso autenticado para salvaguardar la integridad de los datos de los ciudadanos.

## **6. Pruebas**

- Pruebas funcionales del ciclo completo del reporte ciudadano.
- Casos de prueba automatizados bajo el formato Given-When-Then.
- Validación de endpoints de las APIs de integración de mapas e IA.

## **7. DevOps**

- Repositorio GitHub organizado modularmente por responsabilidades de capas.
  - README documentado con la guía de instalación, arquitectura y evidencias del sistema.
-

## **1. Repositorio GitHub**

Link: <https://github.com/Alessandro-BS/SafeDistrict-Diseno>

## **2. Video de funcionamiento**

Link: [https://youtu.be/16jx\\_eyBkJY](https://youtu.be/16jx_eyBkJY)